ENTERPRISE DIGITAL BANKING SYSTEM

06

Testing Documentation

Test strategy and plan, test cases, unit / repository / controller / integration / API tests, coverage and performance testing.

Document	06 — Testing Documentation
System	Enterprise Digital Banking System (banking-app) + notification-service
Stack	Java 17 · Spring Boot 4.1 · Spring Security + JWT · Spring Data JPA · MySQL 8 · Apache Kafka 3.8 · Redis 7 · Docker
Repository	Enterprise-Digital-Banking-System (branch `master`)
Version / date	1.0 — 13 October 2026
Author	hemantwint-hey

Status tags used throughout: **BUILT** exists in the code today **PLANNED** part of the target design (roadmap docs 01–09), not yet implemented **GAP** known limitation

Contents

1. 1. Current State
2. 2. Test Strategy
3. 3. Test Plan
4. 4. Test Cases — Existing
5. 5. Test Cases — Planned
6. 6. Integration and API Tests
7. 7. Coverage
8. 8. Performance Testing

1. Current State

> **Status on 13 Oct 2026: the test sources do not compile.** After the Kafka producers were added, `TransactionServiceImpl` takes a 7th constructor argument (`TransactionEventProducer`) and `BeneficiaryServiceImpl` a 6th (`BeneficiaryEventProducer`). Three test classes still call the old constructors:
>
> - `TransactionServiceImplTest.java:67`
> - `AccountingLogicTest.java:65`
> - `BeneficiaryServiceImplTest.java:58`
>
> **Fix:** add `@Mock TransactionEventProducer transactionEventProducer;` / `@Mock BeneficiaryEventProducer beneficiaryEventProducer;` and pass it as the last constructor argument. Then add a `verify(...).publish(any())` assertion to the transfer and activate success tests.

Test class	Type	Tests	Status
`TransactionServiceImplTest`	Unit (Mockito)	15	DOES NOT COMPILE
`BeneficiaryServiceImplTest`	Unit (Mockito)	15	DOES NOT COMPILE
`AccountingLogicTest`	Unit — ledger invariants with real `LedgerServiceImpl`	2	DOES NOT COMPILE
`LedgerServiceImplTest`	Unit (Mockito)	5	Compiles; passed on its last run, but `./mvnw test` cannot run it until the other three compile
`EnterpriseDigitalBankingSystemApplicationTests`	`@SpringBootTest` context load	1	Needs running MySQL + env vars; fails in isolation
Total		**38**	

notification-service has its own suite (7 classes): `NotificationServiceTest` , `NotificationEngineTest` , `NotificationChannelResolverTest` , `NotificationDecoratorTest` , `NotificationEventPublisherTest` , `ExponentialBackoffRetryPolicyTest` , `JpaNotificationRepositoryTest` .

2. Test Strategy

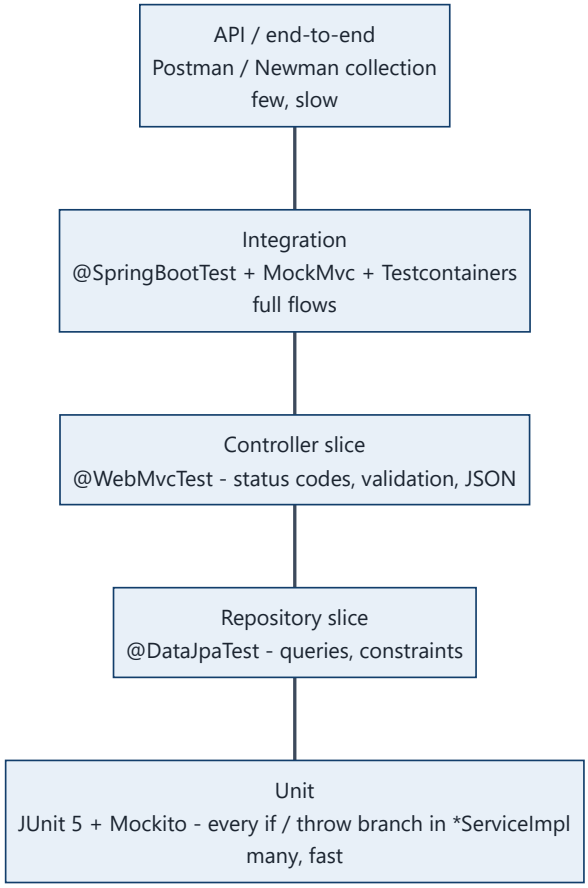


Figure 1 — Test pyramid: most tests at the bottom

Level	Tooling	Scope	Runs in
Unit	JUnit 5, Mockito (`@ExtendWith(MockitoExtension.class)`), AssertJ	One service class; collaborators mocked	ms, no Spring
Repository	`@DataJpaTest` , H2 (test scope) or Testcontainers MySQL	Derived queries, unique / FK constraints, `@Version`	seconds
Controller	`@WebMvcTest` , MockMvc, `@MockitoBean` services, `@AutoConfigureMockMvc(addFilters = false)` or a test JWT	URL mapping, validation → 400, exception handler → JSON	seconds
Integration	`@SpringBootTest(RANDOM_PORT)` , MockMvc, Testcontainers (MySQL, Kafka), `@EmbeddedKafka`	Register → login → … full business flows, security filter	tens of seconds
API / E2E	Postman collection + Newman in CI	Deployed app over HTTP	minutes
Performance	k6 or JMeter	Throughput, latency, concurrency	on demand

2.1 Priorities

1. Fix compilation (section 1).
2. Unit tests for the newest, least-proven logic: `LoanServiceImpl` , `EmiServiceImpl` .
3. `AccountServiceImpl` , `CustomerServiceImpl` , `UserServiceImpl` .
4. Repository tests for custom queries; optimistic-locking test.
5. Controller tests for status codes; one integration test per major flow.

Coverage targets (JaCoCo): service layer ≥ 85 % line / 75 % branch; overall ≥ 70 %. Do not chase 100 % on mappers/DTOs.

3. Test Plan

Item	Plan
Objective	Prove money is never created or lost, business rules are enforced, and every API returns the documented status codes
In scope	All services and endpoints of banking-app; Kafka publishing; notification consumption
Out of scope	Real SMTP delivery, AWS infrastructure
Entry criteria	Code compiles; `./mvnw test-compile` green
Exit criteria	All tests green, coverage targets met, no open high-severity defects, manual smoke test of section 7 passed
Test data	Builders in test classes (`Account.builder()...`); integration tests create their own users via the API; no shared fixtures
Environment	Local JDK 17; Docker for Testcontainers; CI on GitHub Actions
Commands	`./mvnw test` · `./mvnw test -Dtest=LoanServiceImplTest` · `./mvnw verify` (+ JaCoCo report in `target/site/jacoco/index.html`)
Reports	`target/surefire-reports/*.txt` , JaCoCo HTML
Defect handling	GitHub issue per defect with steps, expected/actual; write a failing test first, then fix

4. Test Cases — Existing

4.1 TransactionServiceImplTest

Test	Verifies
deposit_success_creditsAccountAndRecordsLedgerEntry	Balance increases; ledger recorded
deposit_accountNotFound_throwsResourceNotFoundException	404 path
deposit_frozenAccount_throwsBadRequestExceptionAndSkipsLedger	Status check; no ledger write
withdraw_success_debitsAccountAndRecordsLedgerEntry	Balance decreases; ledger
withdraw_insufficientBalance_throwsBadRequestExceptionAndSkipsLedger	Balance guard
transfer_success_movesFundsAndRecordsDoubleEntry	Both balances; ledger
transfer_sameSourceAndDestination_throwsBadRequestException	Self-transfer guard
transfer_insufficientBalance_throwsBadRequestExceptionAndLeavesBalancesUntouched	No partial update
transfer_closedDestinationAccount_throwsBadRequestException	Destination status
transfer_viaActiveBeneficiary_movesFundsToBeneficiaryAccount	Beneficiary resolution
transfer_withBothDestinationAccountIdAndBeneficiaryId_throwsBadRequestException	Exactly-one rule
transfer_withNeitherDestinationAccountIdNorBeneficiaryId_throwsBadRequestException	Exactly-one rule
transfer_viaPendingBeneficiary_throwsBadRequestException	Beneficiary must be ACTIVE
transfer_viaBeneficiaryOwnedByAnotherCustomer_throwsBadRequestException	Ownership
transfer_beneficiaryNotFound_throwsResourceNotFoundException	404 path

4.2 LedgerServiceImplTest and AccountingLogicTest

Test	Verifies
recordTransactionEntries_deposit_createsSingleCreditEntry	1 CREDIT
recordTransactionEntries_withdrawal_createsSingleDebitEntry	1 DEBIT
recordTransactionEntries_transfer_createsBalancedDebitAndCreditEntries	DEBIT + CREDIT of equal amount
getLedgerByAccount_returnsMappedResponses	Mapping
getLedgerByTransaction_whenNoEntriesFound_throwsResourceNotFoundException	404 path
transfer_producesBalancedDoubleEntry_debitsEqualCredits	Σ debits = Σ credits
deposit_thenWithdrawal_netLedgerMovementMatchesFinalBalance	Ledger reconciles to balance

4.3 BeneficiaryServiceImplTest

add (valid, customer missing, account missing, duplicate) · getById (found, not found) · getByCustomer (exists, missing) · update (found, not found) · activate (pending, already active, not found) · delete (found, not found) — 15 tests.

5. Test Cases — Planned

5.1 LoanServiceImplTest / EmiServiceImplTest

ID	Scenario	Expected
LN-01	Apply: 100 000 at 12 % for 12 months	emiAmount = 8884.88, status PENDING
LN-02	Apply: unknown customer / account	ResourceNotFoundException
LN-03	Approve PENDING loan	account balance + principal; status ACTIVE; approvedAt set; `emiService.generateSchedule` called once
LN-04	Approve ACTIVE / REJECTED / CLOSED loan	BadRequestException; balance unchanged; no schedule
LN-05	Reject PENDING loan	REJECTED; no disbursement
LN-06	Reject non-PENDING	BadRequestException
LN-07	getLoansByCustomerId for unknown customer	ResourceNotFoundException
EMI-01	generateSchedule tenure 12	12 rows, installments 1..12, due dates +1..+12 months, all PENDING, amount = emiAmount
EMI-02	Pay PENDING EMI with others unpaid	PAID, paidAt set, loan stays ACTIVE
EMI-03	Pay last unpaid EMI	Loan CLOSED, closedAt set
EMI-04	Pay already-PAID EMI	BadRequestException
EMI-05	Pay unknown EMI	ResourceNotFoundException
EMI-06	Pay EMI of a CLOSED/REJECTED loan	BadRequestException (rule to add)

5.2 AccountServiceImplTest

ID	Scenario	Expected
AC-01	Create for existing customer	Account number matches `^10\d{10}$` ; ACTIVE; holder name = customer name; audit CREATE
AC-02	Generated number collides once	Regenerated (existsByAccountNumber true then false)
AC-03	Freeze already FROZEN	BadRequestException
AC-04	Close with non-zero balance	BadRequestException
AC-05	Update / freeze / close a CLOSED account	BadRequestException
AC-06	Close with zero balance	CLOSED; audit old/new status

5.3 CustomerServiceImplTest and UserServiceImplTest

ID	Scenario	Expected
CU-01	Create with unknown e-mail	UserNotFoundException
CU-02	Second profile for same user / duplicate phone	UserAlreadyExistsException
CU-03	KYC with PAN used by another customer	UserAlreadyExistsException
CU-04	KYC success	VERIFIED; user.kycVerified = true
CU-05	Update keeping own phone number	Allowed (filter excludes self)
US-01	Register duplicate e-mail	UserAlreadyExistsException

US-02	Register password mismatch	InvalidCredentialsException
US-03	Register success	Password stored encoded; role CUSTOMER; audit CREATE
US-04	Login wrong password / unknown e-mail	Exception + LOGIN_FAILED audit
US-05	Login success	Token returned; JwtUtil.extractEmail(token) = e-mail

5.4 Repository, controller and concurrency tests

ID	Test	Expected
RP-01	`LoanRepository.findByCustomerId`	Only that customer's loans
RP-02	`EmiRepository.findByLoanIdOrderByInstallmentNumberAsc`	Ascending order
RP-03	`TransactionRepository.findBySourceAccountIdOrDestinationAccountIdOrderByCreatedAtDesc`	Both directions, newest first
RP-04	Duplicate `account_number` insert	`DataIntegrityViolationException`
RP-05	Optimistic lock: two copies of the same account saved	Second `saveAndFlush` throws `ObjectOptimisticLockingFailureException`
WC-01	`POST /api/v1/loans` valid body	201
WC-02	`POST /api/v1/transactions/deposit` amount 0	400, error contains "amount:"
WC-03	Service throws ResourceNotFoundException	404 JSON with path
WC-04	Protected endpoint without token (filters on)	401 "Authentication required..."

```
@ExtendWith(MockitoExtension.class)
class LoanServiceImplTest {
    @Mock LoanRepository loanRepository;      @Mock CustomerRepository customerRepository;
    @Mock AccountRepository accountRepository; @Mock EmiService emiService;
    @Spy  LoanMapper loanMapper = new LoanMapper();
    @InjectMocks LoanServiceImpl loanService;

    @Test
    void approveLoan_whenPending_disbursesAndGeneratesSchedule() {
        Account acc = Account.builder().id(1L).accountNumber("1000000001").balance(new BigDecimal("500.00")).build();
        Loan loan = Loan.builder().id(9L).customer(Customer.builder().id(1L).build()).disbursementAccount(acc)
                .principalAmount(new BigDecimal("10000.00")).status(LoanStatus.PENDING).build();
        when(loanRepository.findById(9L)).thenReturn(Optional.of(loan));
        when(loanRepository.save(any())).thenAnswer(i -> i.getArgument(0));

        LoanResponse res = loanService.approveLoan(9L);

        assertThat(res.getStatus()).isEqualTo(LoanStatus.ACTIVE);
        assertThat(acc.getBalance()).isEqualByComparingTo("10500.00");
        verify(emiService).generateSchedule(loan);
    }
}
```

6. Integration and API Tests

6.1 Integration flows PLANNED

ID	Flow	Assertions
IT-01	Register → login → create customer → KYC → open 2 accounts → transfer	Balances, 2 ledger rows, audit rows, 200s
IT-02	Apply loan → approve → list EMIs → pay all	N EMIs; loan CLOSED after last payment
IT-03	Add beneficiary → transfer (PENDING) fails → activate → transfer succeeds	400 then 200; event on `beneficiary.activated` (`@EmbeddedKafka` consumer)
IT-04	Call any `/api/v1/**` without / with expired token	401 JSON
IT-05	Kafka broker down during transfer	Transfer still 200 (fire-and-forget)
IT-06	Redis cache: GET account twice, deposit, GET again	Second GET no SQL; after deposit new balance returned

6.2 API test collection

A Postman collection mirroring document 04 §11 (quick-start) with tests per request (`pm.response.to.have.status(201)`, save `token`, `accountId` into collection variables). Run in CI with `newman run banking.postman_collection.json -e local.json` PLANNED.

6.3 Manual cross-service test (Kafka live)

1. Start Kafka (`docker compose up -d` in notification-service), banking-app (8080), notification-service (8081).
2. Register, log in, create a customer and two accounts, POST a transfer.
3. notification-service log shows the `transaction.completed` event consumed.
4. `GET :8081/api/v1/notifications?userId=<uuid>` → a row with status SENT (or FAILED while SMTP credentials are unset — expected).
5. Repeat with `PATCH /beneficiaries/{id}/activate` for `beneficiary.activated`.

7. Coverage

```
<!-- pom.xml (planned) -->
<plugin>
  <groupId>org.jacoco</groupId><artifactId>jacoco-maven-plugin</artifactId><version>0.8.12</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>report</id><phase>verify</phase><goals><goal>report</goal></goals></execution>
  </executions>
</plugin>
```

Package	Covered today (by design of existing tests)	Target
transaction.service	All branches of deposit / withdraw / transfer (once compiling)	≥ 90 %
ledger.service	All entry types	≥ 90 %
beneficiary.service	All public methods, happy + error paths	≥ 90 %
loan.service, emi.service	none	≥ 85 %
account.service, customer.service, auth.service	none	≥ 80 %
security, exception	none	≥ 70 % via controller/integration tests

Coverage percentages are produced by JaCoCo once the plugin is added; this document will be updated with measured numbers.

8. Performance Testing

8.1 Goals

Scenario	Load	Target
Login	20 req/s	p95 < 300 ms (BCrypt dominates)
GET account	200 req/s	p95 < 50 ms with Redis
Transfer	50 req/s across 100 accounts	p95 < 250 ms, 0 % 5xx, balances reconcile
Transfer contention	20 concurrent transfers from the same account	No lost update; losers receive a conflict; sum of balances unchanged
History	account with 10 000 transactions	paged response p95 < 100 ms (requires pagination + indexes)

8.2 k6 script (sketch)

```
import http from 'k6/http';
import { check } from 'k6';
export const options = { vus: 50, duration: '2m', thresholds: { http_req_duration: ['p(95)<250'] } };
const H = { headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${__ENV.TOKEN}` } };
export default function () {
  const src = 1 + Math.floor(Math.random() * 50), dst = 51 + Math.floor(Math.random() * 50);
  const r = http.post('http://localhost:8080/api/v1/transactions/transfer',
      JSON.stringify({ sourceAccountId: src, destinationAccountId: dst, amount: 1 }), H);
  check(r, { 'status 200': (x) => x.status === 200 });
}
```

8.3 What to watch

- **N+1 queries**: with `show-sql=true`, history of an account with many transactions should not issue one query per row; fix with `@EntityGraph` / `JOIN FETCH`.
- **HikariCP** pending connections (default pool 10).
- After the run, the reconciliation query in document 07 §9.1 must return no rows and total money across accounts must be unchanged.